



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutierrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Bryant Maximiliano Dzul Perez

Fechas
09/Junio/2026

Introducción

El desarrollo de software actual es complejo pues los sistemas han crecido de forma exponencial, exigiendo metodologías que vayan más allá de la simple escritura de código lineal.

Al aumentar las dimensiones y funcionalidades, se hace necesario utilizar estructuras y enfoques organizativos que permitan ordenar de mejor manera el código fuente, separar de forma clara las responsabilidades de cada componente y facilitar el mantenimiento continuo de las plataforma a los largo de su ciclo de vida.

Para ello, la siguiente investigación permitirá comparar escalabilidad, mantenibilidad, modularidad, etc, de distintos patrones de arquitectura para analizar y comparar detalladamente las arquitecturas más utilizadas en la actualidad con el fin de consolidar un criterio técnico sólido para seleccionar la estructura idónea según las necesidades reales de cada proyecto.

Patrones de Arquitectura de Software

1. MVC (Model-View-Controller)

Definición: Es un patrón de diseño arquitectónico que separa la información de una aplicación, la interfaz de usuario y la lógica de control en tres componentes distintos.

Estructura general: Triádica e interconectada, diseñada para aislar la lógica de negocio de las consideraciones de la interfaz.

Componentes principales: El **Modelo** (gestiona los datos y las reglas de negocio), la **Vista** (la interfaz gráfica que el usuario ve) y el **Controlador** (gestiona las peticiones del usuario e interactúa con el Modelo).

Forma de funcionamiento: El usuario realiza una acción en la Vista. El Controlador capta esta acción, procesa la petición y actualiza el Modelo si es necesario. Posteriormente, el Controlador indica a la Vista que se actualice para reflejar los nuevos datos.

Ventajas: Facilita el desarrollo en paralelo, permite actualizar la interfaz sin modificar la lógica y hace que el código sea más ordenado y fácil de mantener.

Desventajas: Puede presentar una curva de aprendizaje inicial pronunciada y, en aplicaciones muy grandes, los controladores pueden sobrecargarse de responsabilidades.

Casos de uso: Plataformas web modernas, sistemas de gestión escolar y aplicaciones que utilizan frameworks backend robustos como Laravel o Django.

2. Arquitectura en Capas (Layered Architecture)

Definición: Es un patrón que organiza un sistema de software en divisiones jerárquicas (capas), donde cada capa provee servicios específicos a la capa inmediatamente superior.

Estructura general: Apilada, formando una estructura de "n-niveles" (n-tier) donde la comunicación fluye secuencialmente.

Componentes principales: Típicamente consta de la capa de Presentación (UI), capa de Lógica de Negocio (reglas operativas), capa de Acceso a Datos (persistencia) y la Base de Datos.

Forma de funcionamiento: Las peticiones del usuario entran por la capa superior y atraviesan secuencialmente las capas inferiores hasta llegar a los datos. La respuesta hace el recorrido inverso. Una capa no puede saltarse a la siguiente para comunicarse con otra más profunda.

Ventajas: Alta cohesión, fácil estandarización, promueve la separación de responsabilidades y simplifica las pruebas unitarias por capa.

Desventajas: Puede generar latencia debido a que las peticiones deben atravesar múltiples capas. Además, cambios simples en la base de datos suelen requerir modificaciones en todas las capas superiores.

Casos de uso: Aplicaciones empresariales corporativas, sistemas ERP tradicionales y software de escritorio complejo.

3. Arquitectura Monolítica

Definición: Un modelo tradicional donde todos los aspectos funcionales del sistema (interfaz, lógica y datos) están entrelazados y unificados en un único programa ejecutable.

Estructura general: Un bloque único, centralizado e indivisible.

Componentes principales: Interfaz de usuario, capa de servicios y acceso a datos consolidados dentro de un mismo repositorio de código.

Forma de funcionamiento: Todas las funciones operan dentro del mismo proceso y espacio de memoria del servidor, comunicándose internamente de forma directa y accediendo a una base de datos compartida.

Ventajas: Es fácil de desarrollar en etapas iniciales, simplifica las pruebas de integración y el despliegue es muy directo al manejar un solo artefacto.

Desventajas: A medida que la aplicación crece, el código se vuelve difícil de mantener y entender. Un fallo en un módulo puede hacer colapsar toda la aplicación, y escalar implica duplicar todo el sistema.

Casos de uso: Proyectos en fases tempranas, pruebas de concepto (MVP) o plataformas de gestión interna con tráfico moderado y controlado.

4. Microservicios

Definición: Un enfoque que divide una aplicación compleja en un conjunto de servicios pequeños, independientes y autónomos que se comunican entre sí a través de la red.

Estructura general: Descentralizada y distribuida, formando una red de pequeños componentes colaborativos.

Componentes principales: Servicios independientes (ej. servicio de autenticación, servicio de facturación, bases de datos aisladas por servicio y un API Gateway (puerta de enlace) para enrutar las peticiones.

Forma de funcionamiento: Cada servicio ejecuta su propio proceso y gestiona su propia base de datos. Se comunican entre ellos utilizando protocolos ligeros como HTTP/REST o mensajería asíncrona.

Ventajas: Escalabilidad extremadamente precisa (se escala solo lo que se necesita), alta tolerancia a fallos y permite usar diferentes tecnologías de programación para diferentes servicios.

Desventajas: La infraestructura es altamente compleja de configurar y mantener. Requiere orquestación avanzada, gestión de redes compleja y hace muy difíciles las pruebas end-to-end.

Casos de uso: Plataformas masivas a nivel global (como Netflix o Amazon), e-commerce de alto tráfico y sistemas que requieren alta disponibilidad.

5. Cliente-Servidor

Definición: Es un modelo de arquitectura distribuida en el que las tareas se reparten entre los proveedores de recursos o servicios (servidores) y los demandantes (clientes).

Estructura general: Topología en forma de estrella lógica, centralizada en torno a un proveedor y múltiples nodos consumidores.

Componentes principales: El **Cliente** (aplicación web, móvil o de escritorio del usuario), el **Servidor** (máquina o clúster que procesa) y una **Red** que facilita la comunicación entre ambos.

Forma de funcionamiento: El cliente inicia una solicitud a través de la red hacia el servidor. El servidor procesa la petición, accede a los recursos necesarios (archivos, bases de datos) y devuelve el resultado al cliente, quien lo muestra al usuario.

Ventajas: Permite una gestión centralizada de la seguridad y los datos, facilita el control de concurrencia y libera a los clientes de la carga de procesamiento pesado.

Desventajas: El servidor se convierte en un punto único de fallo (si cae, todos los clientes quedan inoperativos) y puede convertirse en un cuello de botella si el tráfico es muy alto.

Casos de uso: Sistemas de correo electrónico, bases de datos centralizadas, servicios de impresión en red y la arquitectura base de la World Wide Web.

Aspecto	MVC (Model-View-Controller)	Arquitectura en Capas	Monolítica	Microservicios	Cliente-Servidor
Objetivo principal	Separar la lógica de negocio, la interfaz y el control de flujo.	Organizar el sistema en niveles jerárquicos con responsabilidades específicas.	Unificar todas las funciones del sistema en una sola base de código y despliegue.	Dividir el sistema en servicios pequeños, independientes y comunicados por red.	Separar a los proveedores de recursos (servidores) de los demandantes (clientes).
Estructura	Tres componentes: Modelo (Datos), Vista (UI) y Controlador (Lógica).	Capas apiladas (Presentación, Negocio, Persistencia, Base de Datos).	Un único bloque ejecutable que contiene la UI, lógica y acceso a datos.	Múltiples servicios independientes, cada uno con su propia base de datos.	Dos nodos principales: Frontend (Cliente) y Backend (Servidor).
Organización del código	Alta cohesión; código agrupado por función técnica.	Separación estricta por responsabilidades; una capa solo habla con la adyacente.	Centralizada; todo el código reside en un mismo repositorio.	Distribuida; cada servicio tiene su propio repositorio y entorno.	Dividida físicamente entre las aplicaciones del usuario y el servidor central.

Escalabilidad	Moderada (generalmente se escala toda la aplicación junta).	Moderada (se debe escalar la aplicación completa, no capas individuales).	Baja (requiere replicar toda la aplicación, consumiendo muchos recursos).	Muy Alta (permite escalar solo los servicios que tienen alta demanda).	Alta (el servidor se puede escalar vertical u horizontalmente).
Mantenimiento	Facilita actualizaciones en la UI sin tocar la lógica.	Sencillo si se respetan las reglas de aislamiento entre capas.	Difícil a medida que el código crece (código espagueti).	Complejo a nivel de infraestructura, pero fácil a nivel de código por servicio.	Centralizado en el servidor; los clientes requieren actualizaciones menos críticas.
Complejidad	Baja a Media.	Media.	Baja (al inicio) a Alta (al crecer).	Muy Alta (requiere orquestación, redes, CI/CD avanzado).	Media.
Facilidad de implementación	Muy alta, soportada por casi todos los frameworks web modernos.	Alta, es el estándar tradicional para aplicaciones empresariales.	Muy alta, ideal para iniciar proyectos rápidamente.	Baja, requiere una curva de aprendizaje pronunciada e infraestructura robusta.	Alta, estándar en la mayoría de aplicaciones web y móviles.

Rendimiento	Bueno, depende del framework y la optimización del controlador.	Puede sufrir latencia si hay demasiadas capas que atravesar.	Alto en sistemas pequeños (comunicación en memoria rápida).	Variable (la comunicación por red entre servicios puede generar latencia).	Depende del ancho de banda de la red y la capacidad del servidor.
Reutilización	Alta (modelos y vistas pueden reutilizarse fácilmente).	Alta (la capa de negocio puede servir a diferentes interfaces).	Baja (difícil extraer módulos para otros proyectos).	Muy Alta (los servicios pueden ser consumidos por múltiples aplicaciones).	Media (los servicios del servidor son reutilizables por múltiples clientes).
Casos de uso recomendados	Aplicaciones web, interfaces de usuario interactivas, CMS.	Sistemas empresariales tradicionales, aplicaciones de escritorio complejas.	Proyectos pequeños, pruebas de concepto, equipos reducidos.	Sistemas masivos (Netflix, Amazon), alta escalabilidad requerida.	Bases de datos centralizadas, correo electrónico, aplicaciones web tradicionales.
Ventajas	Desarrollo en paralelo, fácil prueba de componentes.	Aislamiento funcional, fácil de entender y estandarizar.	Fácil de desarrollar, probar y desplegar al inicio.	Resiliencia ante fallos, independencia tecnológica por servicio.	Seguridad centralizada, fácil gestión de datos concurrentes.

Desventajas	Puede volverse complejo si los controladores engordan mucho.	Rigidez (los cambios suelen afectar a múltiples capas).	Acoplamiento fuerte, despliegues lentos en sistemas grandes.	Difícil monitoreo, pruebas de integración complejas, consistencia de datos.	Si el servidor cae, los clientes quedan inoperativos (cuello de botella).
--------------------	--	---	--	---	---

Análisis

1. ¿Cuál de los patrones consideras más adecuado para una plataforma web desarrollada por un equipo pequeño y por qué?

El patrón MVC (Model-View-Controller). Para un equipo pequeño, la prioridad es la velocidad de desarrollo, la simplicidad en el despliegue. Utilizar frameworks robustos basados en MVC permiten a un equipo reducido crear sistemas completos de manera ordenada y estandarizada.

2. ¿Qué ventajas ofrecen los microservicios frente a una arquitectura monolítica?

Los microservicios ofrecen escalabilidad independiente y flexibilidad tecnológica. En un monolito, un error grave puede corromper todo el sistema.

3. ¿Por qué MVC continúa siendo uno de los patrones más utilizados en aplicaciones web?

Porque mapea de forma natural cómo funciona la web moderna, por ejemplo, el usuario hace una petición (Controlador), la aplicación consulta la base de datos (Modelo) y devuelve un HTML o JSON (Vista). Además, separar las responsabilidades, permite que los desarrolladores Frontend y Backend trabajen en paralelo.

4. ¿Qué riesgos podrían existir al implementar una arquitectura demasiado compleja para un proyecto pequeño?

El principal riesgo es la sobreingeniería. Implementar microservicios para un proyecto pequeño multiplica los costos de servidores, ralentiza el desarrollo inicial, complica el despliegue y consume el tiempo del equipo resolviendo problemas de infraestructura en lugar de programar la lógica del negocio.

5. ¿Crees que existe un patrón mejor que todos los demás? Justifica tu respuesta.

No, ya que depende del contexto del proyecto. Un monolito valida una idea de negocio rápido, los microservicios son indispensables para proyectos grandes y MVC es el estándar para el desarrollo web.

Caso

Sistema seleccionado: Sistema escolar

Patrón de arquitectura seleccionado: MVC (Model-View-Controller) con enfoque monolítico.

Justificación:

Número de usuarios: Un sistema escolar tiene un número de usuarios predecible y limitado (alumnos, profesores y administrativos). No requiere la concurrencia masiva de una plataforma como Netflix.

Escalabilidad: Es moderada. Los picos de tráfico ocurren en momentos muy específicos. Un monolito bien optimizado en un servidor en la nube puede manejar estos picos escalando verticalmente de forma temporal.

Complejidad: Un sistema escolar incluye módulos claros (calificaciones, roles, directorios, autenticación). El patrón MVC permite separar perfectamente la lógica de la base de datos, la interfaz para alumnos/profesores y las reglas de negocio de la institución.

Mantenimiento: Al estar todo centralizado, las actualizaciones a las reglas académicas son más fáciles de auditar y mantener en un solo repositorio.

Costos: Es sumamente rentable. Solo se requiere un servidor web y un servidor de base de datos, manteniendo los costos de infraestructura al mínimo para la institución.

Tamaño del equipo: Puede constar del equipo de TI de la institución o bien de un equipo pequeño de desarrolladores, aproximadamente 3 a 5 personas.

Conclusión

Comprendí que los patrones de arquitectura son planos fundamentales que indican cómo se comunicarán los módulos del software. No existe una solución única.

El patrón MVC me resulta más interesante pues es el que más se adapta al desarrollo web, permitiendo crear sistemas complejos manteniendo el código limpio y organizado-

La arquitectura define el límite del software, tomar malas decisiones puede hacer que el código se vuelva un caos imposible de mantener con el tiempo, mientras que una buena arquitectura asegura que el sistema sea estable, seguro y capaz de escalar sin romperse.

Es importante tomar una buena decisión desde el inicio ya que cambiar la arquitectura cuando el software ya está construido es un proceso costoso y riesgoso en el desarrollo de sistemas.